



华中科技大学

循环程序

《x86汇编语言程序设计》第7章，许向阳，华科大学出版社，2020年





华中科技大学

第7章 循环程序设计

7.1 循环程序的结构

7.2 C循环语句的反汇编





课堂目标

- 能够给出 C 循环语句的 机器级实现方式
- 对于一段含循环的机器指令，能够指出其功能
- 能够用机器语言对循环程序进行**优化**
 - 变量与寄存器绑定**，减少读写变量的次数；
 - 循环展开**，即消除循环，提高指令流水线的利用率；
 - 使用更大的寄存器**、成组运算等，减少循环次数；
 - 在编译期间，能够求出结果的，直接给出结果，而不再生成对应的机器指令。





C循环语句的反汇编

```
int i = 0, sum = 0, a[5];
```

```
.....
```

```
for (i = 0; i < 5; i++)    sum += a[i];
```

```
00D71750  mov     dword ptr [i], 0
00D71757  jmp     f+62h (0D71762h)
00D71759  mov     eax, dword ptr [i]
00D7175C  add     eax, 1
00D7175F  mov     dword ptr [i], eax
00D71762  cmp     dword ptr [i], 5
00D71766  jge     f+77h (0D71777h)
00D71768  mov     eax, dword ptr [i]
00D7176B  mov     ecx, dword ptr [sum]
00D7176E  add     ecx, dword ptr a[eax*4]
00D71772  mov     dword ptr [sum], ecx
00D71775  jmp     f+59h (0D71759h)
00D71777  // 循环结束
```

采用分支转移指令可实现循环。





7.1 循环程序的结构

循环控制方法

计数控制：循环次数已知时常用

(1) 倒计时

.....

MOV ECX, 循环次数

LOOPA:

.....

DEC ECX

JNE LOOPA

Q: 能否用其他寄存器或者变量控制循环次数?

- 循环次数n → 循环计数器
- 每循环一次，计数器减1
- 直到计数器值为0时，结束循环





7.1 循环程序的结构

循环
控制
方法

计数控制：循环次数已知时常用
(2) 正计数

.....

MOV ECX, 0

LOOPA:

.....

INC ECX

CMP ECX, n

JNE LOOPA





7.1 循环程序的结构

循环控制方法

条件控制：循环次数不固定

通过指令来测试条件是否成立，
决定继续循环还是结束循环。

例：求一个以0为结束符的字符串的长度





7.1 循环程序的结构

循环
控制
方法

阅读程序段，指出其功能：

```
        MOV    CL, 0
L:      AND    AX , AX
        JZ     EXIT
        SAL    AX , 1
        JNC    L
        INC    CL
        JMP    L
EXIT:
```

(AX) 中 1 出现的次数 → CL





7.1 循环程序的结构

循环
控制
方法

阅读程序段，指出其功能：

```
        MOV     CL,  0
        MOV     BX, 16
L:       SAL     AX , 1
        JNC     NEXT
        INC     CL
NEXT:    DEC     BX
        JNZ     L
```

(AX) 中 1 出现的次数 → CL





7.1 循环程序的结构

80X86提供的四种**计数控制**循环转移指令

**循环
控制
指令**

LOOP	标号
LOOPE	标号
LOOPNE	标号
JECXZ	标号

(1) **LOOP** 标号

(ECX) -1 → ECX

若 (ECX) 不为0, 则转标号处执行。

基本等价于: DEC ECX
 JNZ 标号

(LOOP指令对标志位无影响 !)





7.1 循环程序的结构

(2) LOOPE /LOOPZ 标号

$(ECX) - 1 \rightarrow ECX$

若 (ECX) 不为0, 且 $ZF=1$, 则转标号处执行。

(等于或为0循环转移指令, 本指令对标志位无影响)

32位段用 ECX , 16位段用 CX





7.1 循环程序的结构

(2) LOOPE /LOOPZ 标号

例：判断以BUF为首址的10个字节中是否有非0字节。
有，则置ZF为0，否则ZF置为1。

```
        MOV     ECX, 10
        MOV     EBX, OFFSET BUF -1
L3 :    INC     EBX
        CMP     BYTE PTR [EBX], 0
        LOOPE   L3
```





7.1 循环程序的结构

(3) LOOPNE /LOOPNZ 标号

(ECX) -1 → ECX

若 (ECX) ≠ 0, 且 ZF=0, 则转标号处执行。

例：判断以MSG为首址的10个字节中的串中是否有空格字符。无空格字符，置ZF为0，否则为1。

```
        MOV     ECX,    10
        MOV     EBX,    OFFSET MSG -1
L4 :    INC     EBX
        CMP     BYTE PTR [EBX],    ' '
        LOOPNE  L4
```





7.1 循环程序的结构

(4) JECXZ 标号

若 (ECX) 为0, 则转标号处执行。

(先判断, 后执行循环体时, 可用此语句,
标号为循环结束处)





7.2 C循环语句的反汇编

```
int i = 0, sum = 0, a[5];
```

```
.....
```

```
for (i = 0; i < 5; i++)    sum += a[i];
```

```
00D71750  mov     dword ptr [i], 0
00D71757  jmp     f+62h (0D71762h)
00D71759  mov     eax, dword ptr [i]
00D7175C  add     eax, 1
00D7175F  mov     dword ptr [i], eax
00D71762  cmp     dword ptr [i], 5
00D71766  jge     f+77h (0D71777h)
00D71768  mov     eax, dword ptr [i]
00D7176B  mov     ecx, dword ptr [sum]
00D7176E  add     ecx, dword ptr a[eax*4]
00D71772  mov     dword ptr [sum], ecx
00D71775  jmp     f+59h (0D71759h)
00D71777  // 循环结束
```

Debug 版本

Q: 程序段有多少条语句?

但完成循环, 又要执行多少条语句?

程序段: 12条指令
循环执行指令数 50余条
(10*5+2+...)

Q: 可以做哪些优化?

变量与寄存器绑定!
循环展开!





7.2 C循环语句的反汇编

Release 编译优化

```
int i = 0, sum = 0, a[5];
```

```
.....
```

```
for (i = 0; i < 5; i++)    sum += a[i];
```

```
mov    eax, dword ptr [ebp-8]
add    eax, dword ptr [ebp-0Ch]
add    eax, dword ptr [ebp-10h]
add    eax, dword ptr [ebp-14h]
add    eax, dword ptr [ebp-18h]
mov    sum, eax
```

Q: 如果循环次数 5 改为一个变量，又如何优化？

```
for(i=0; i<n; i++) sum+=a[i] ;
```





7.2 C循环语句的反汇编

Release 编译优化

```
int i = 0, sum = 0, a[5];
```

.....

```
for (i = 0; i < n; i++)    sum += a[i];
```

```
    mov     edi, sum      ; edi来存放 和
```

```
    xor     eax, eax      ; eax 对应 i
```

```
    mov     edx, n        ; edx 对应 n
```

```
    jmp     main+0C5h (05E1145h)
```

```
005E1140  add     edi, dword ptr [ebp+eax*4-18h]
```

```
005E1144  inc     eax
```

```
005E1145  cmp     eax, edx
```

```
005E1147  jnl     main+0C0h (05E1140h)
```

变量与寄存器绑定，语句数量大幅减少！
循环部分：由 10 条语句减为 4条语句！





7.2 C循环语句的反汇编

Release 编译优化

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

void main()
{
    char  buf1[20];
    char  buf2[20];
    int    i;
    scanf("%s", buf1);
    for (i = 0; i < 20; i++)
        buf2[i] = buf1[i];
    printf("%s\n", buf2);
    return;
}
```





7.2 C循环语句的反汇编

Release 编译优化

```
for (i = 0; i < 20; i++)  
    buf2[i] = buf1[i];  
printf("%s\n", buf2);
```

```
0025109E  mov     eax, dword ptr [ebp-8]  
002510A1  movups  xmm0, xmmword ptr [buf1]  
002510A5  mov     dword ptr [ebp-1Ch], eax  
002510A8  lea     eax, [buf2]  
002510AB  push    eax  
002510AC  push    offset string "%s\n" (0252104h)  
002510B1  movups  xmmword ptr [buf2], xmm0  
002510B5  call    printf (0251020h)
```

buf1[0] 小地址
buf1[1]
.....
buf1[15]
buf1[16] 大地址
.....
buf1[19]

Q: 这段代码如何解读?

buf1 的前16个字节拷贝到 xmm0
后4个字节拷贝到 eax
再分别送到 buf2 相应位置

监视 1	
搜索(Ctrl+E) 🔍 搜索深度: 3	
名称	值
▶ buf1	0x006ffedc "abcdefg"
▶ buf2	0x006ffec8 "abcdefg"
ebp,x	0x006ffef4
ebp-8,x	0x006ffec





7.2 C循环语句的反汇编

Q: 能否写一个C程序, 能实现 buf1 中的内容拷贝到 buf2 中, 但Release又不好优化?

```
void main()
{
    char  buf1[20];
    char  buf2[20];
    int    i;
    scanf("%s", buf1);
    .....
    printf("%s\n", buf2);
    return;
}
```

```
void fcopy(char* dst, char* src)
{
    int i;
    for (i = 0; i < 20; i++)
    {
        *dst = *src;
        dst++;
        src++;
    }

    fcopy(buf2, buf1);
}
```

fcopy(buf1-20, buf1); // 可能存在数据相关, 未优化





7.2 C循环语句的反汇编

scanf("%s", buf1);

003D1090 lea eax, [ebp-18h]

003D1093 push eax

003D1094 push 3D2100h

003D1099 call 003D1050

003D109E add esp, 8

003D10A1 xor eax, eax

fcopy(buf1-20, buf1);

003D10A3 mov cl, byte ptr [ebp+eax-18h]

003D10A7 mov byte ptr [ebp+eax-2Ch], cl

003D10AB inc eax

003D10AC cmp eax, 14h

003D10AF jl 003D10A3

printf("%s\n", buf2);



- 用分支转移指令，实现循环；
- 有专门的循环指令：LOOP、LOOPE、LOOPNE、JECXZ

➤ 编译优化

循环展开：消除了循环

与寄存器绑定：减少访存操作，减少指令

用XMM寄存器、成组运算等，减少循环执行次数

在编译期间计算出结果（或部分结果），减少执行程序的指令